

Solving PDEs using Deep Learning: Poisson Equation on Unit Disk using PINNs

Progress Report 1

Larry Hui

14 February, 2025

Table of Contents

- ▶ Introduction and Objectives
- ▶ Replication of 1D, 2D PINNs based CSM
- ▶ Introductory Papers to PINNs

Onboarding Projects

1 Introduction and Objectives

- In order for me to get a better grasp on the project, some MATLAB tutorials were completed, namely: **1) solving PDE's using deep learning** and **2) solving the Poisson Equation on unit disk using PINNs**.
 - Recreated the 1D and 2D results obtained in Jinshuai Bai et. al's paper on PINNs-based computational mechanics
- I then read through a couple research papers on the subject including:
 - *PINNs: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs*
 - *Solving spatio temporal partial differential equations with Physics-informed GNNs*
 - *A fully differentiable GNN-based PDE Solver: With Applications to Poisson and Navier-Stokes Equations*
 - *An introduction to programming PINNs-based computational solid mechanics*

- ▶ Introduction and Objectives
- ▶ Replication of 2D PINNs based CSM
- ▶ Introductory Papers to PINNs

Replication of 2D results of PINNs CSM

2 Replication of 2D PINNs based CSM

We will consider a 2D in-plane stretching square plate problem. Due to the geometric symmetry, a quarter of the plate is used for the modelling. The length of the plate $L = 2$ m. A distributed force, $t(y)$, is applied on the right side of the plate.

Replication of 2D results of PINNs CSM

2 Replication of 2D PINNs based CSM

- ▶ Introduction and Objectives
- ▶ Replication of 1D, 2D PINNs based CSM
- ▶ **Introductory Papers to PINNs**
- ▶ Takeaways

Summary of Papers

3 Introductory Papers to PINNs

I have read the following papers and their findings are succinctly synthesized and summarized in the following slides:

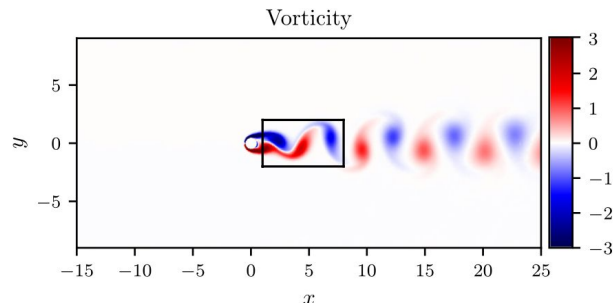
1. *PINNs: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs*
2. *Solving spatio temporal partial differential equations with Physics-informed GNNs*
3. *A fully differentiable GNN-based PDE Solver: With Applications to Poisson and Navier-Stokes Equations*
4. *An introduction to programming PINNs-based computational solid mechanics*

Key Concepts

1. PINNs: framework integrating physical laws into neural networks for solving forward/inverse problems.
2. Applications: Solving PDEs, finding unknown PDEs from data, and modeling physical systems with limited data.

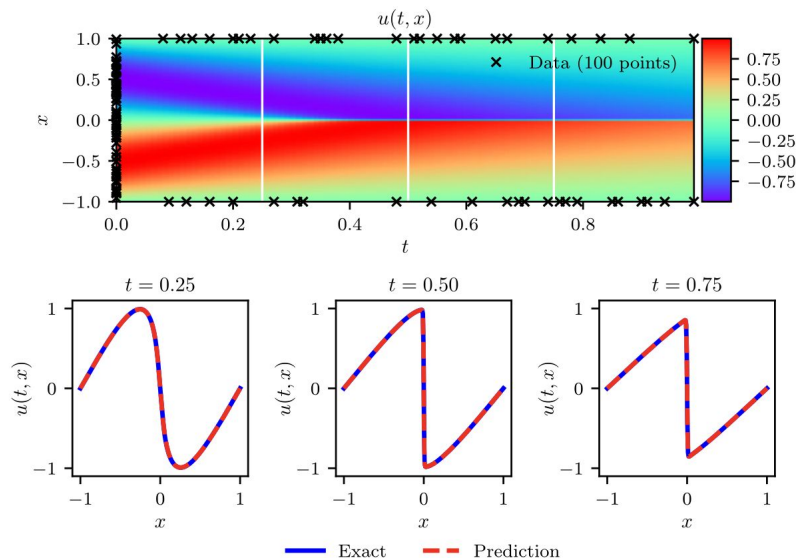
Main Features and Advantages

- *CT Models* (Example: Schrödinger equation solved with high accuracy using limited training data.)
 - Use automatic differentiation to enforce PDE constraints.
 - Loss function combines data fitting and PDE residual minimization.
- *DT Models* (Example: Allen-Cahn equation solved in a single time step with minimal error.)
 - Leverage Runge-Kutta time-stepping schemes for structured learning.
 - Suitable for large time steps and high-dimensional problems.
- *Advantages:* Data efficiency: fewer training points by embedding physics; Robustness to noise; Flexibility: Applicable to nonlinear and high-dimensional PDEs.



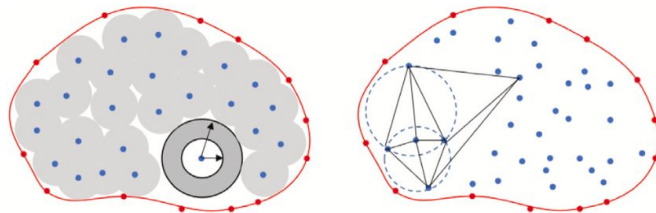
Data-Driven PDE Solutions

- PINNs solve KdV equations, Burger's equation using small data sets. For example, it was determined that the Burgers' equation was solved with a relative L^2 error of 6.7×10^{-4} capturing sharp internal layers



Data-Driven Discovery of PDEs

- PINNs identify unknown parameters and governing equations from sparse or noisy data. N-S equations identified with $<1\%$ error in parameters using only velocity field data.
 - accurately models vortex shedding without pressure data.
- PINNs bridge machine learning and computational physics, enabling efficient, accurate solutions for scientific problems while opening avenues for further research in deep learning for physical systems.



(a) Poisson disc sampling method (b) Bowyer-Watson algorithm

Key Concepts

1. **Physics-Informed GNNs:** Combines GNNs with Radial Basis Function-Finite Difference (RBF-FD) to solve spatiotemporal PDEs on irregular domains. Utilizes MeshGraphNets with Encoder-Processor-Decoder architecture for dynamic state modeling.
2. **RBF-FD Integration:** Meshless method for spatial discretization, enabling accurate PDE residual computation without domain meshing. Handles irregular geometries and scattered node layouts efficiently.

Advantages

- Accurate predictions for complex PDEs like heat, wave, and shallow water equations.
- Strong generalization across different initial conditions, boundary conditions, and noise levels.
- Computational efficiency compared to traditional PINNs on irregular domains.

Graph Construction

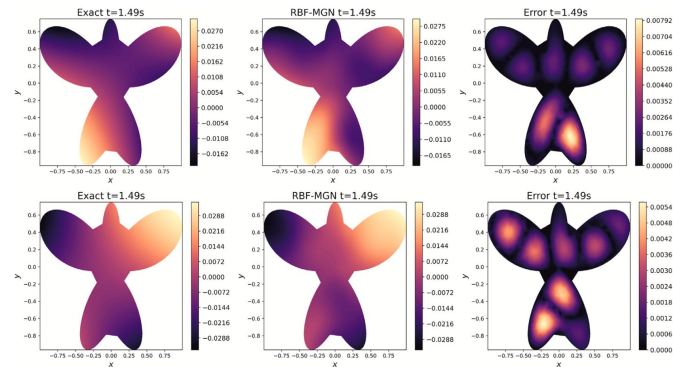
1. Nodes generated via Poisson disc sampling for uniform distribution.
2. Edges constructed using Delaunay triangulation to maintain geometric accuracy.

Paper 2 continued

3 Introductory Papers to PINNs

Experimental Results

- 2D Heat Equation
 - Achieved high accuracy with relative L^2 -error as 6.43×10^{-4} for small time steps $\Delta t = 0.01$; demonstrated ability to extrapolate solutions over time with minimal error accumulation.
- Irregular Domains
 - RBF-MGN outperformed PINNs in accuracy and stability over longer time steps. Errors remained at 10^{-6} level for extended time predictions
- Comparison with PINNs
 - PINNs struggled with error accumulation over time, whereas RBF-MGN maintained consistent accuracy due to hard-constrained PDE residuals



Paper 3

3 Introductory Papers to PINNs

Key Concepts

1. **Framework:** Combines the FVM with GNNs to solve PDEs like Poisson and Navier-Stokes.
2. **Unsupervised Training:** Utilizes GPU parallel computing to implement a fully differentiable FVM, enabling direct PDE solving during training without pre-computed data.
3. **Adaptability:** Handles multiple boundary conditions, source terms, and different PDE types on unstructured 2D grids

Core Features and Advantages

- Differentiable integral and gradient reconstruction algorithms for accurate PDE discretization; Encodes PDE terms as learnable features, allowing the model to generalize across various equations and domains.
- Meshless formulation ensures flexibility for irregular geometries implying superior generalization across mesh types and boundary conditions.
- Eliminates reliance on labeled data, reducing computational costs compared to traditional PINNs.

Paper 3 continued

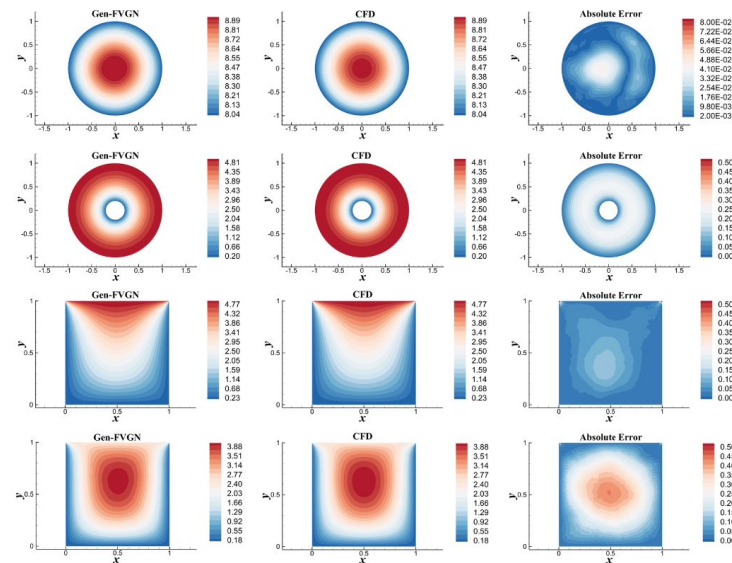
3 Introductory Papers to PINNs

Performance Highlights

1. **Poisson Equation:** Solved with high accuracy on irregular grids, demonstrating robustness in handling mixed boundary conditions.
2. **Navier-Stokes:** Effectively simulated incompressible fluid flow on unstructured grids, showcasing adaptability to complex geometries.

Comparison with PINNs

- Traditional PINNs struggle with fitting accuracy and generalization due to their point-wise structure.
- The proposed GNN-based solver achieves better accuracy and stability by leveraging FVM-inspired loss functions.
- Consistent performance across diverse PDEs without task-specific tuning.
- Demonstrated scalability for CFD applications.



Paper 4

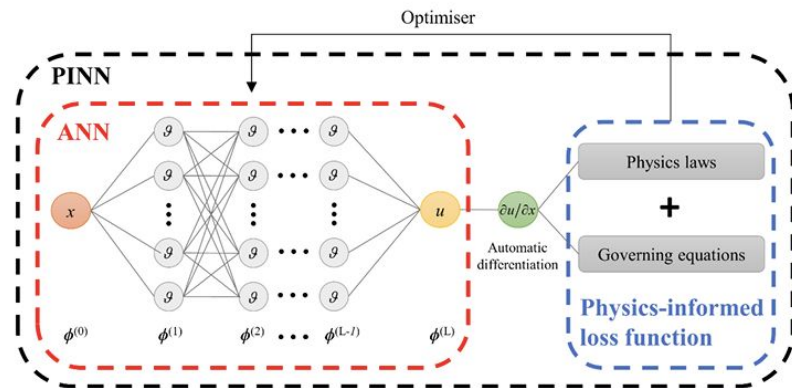
3 Introductory Papers to PINNs

Key Concepts

- Applications in Solid Mechanics
 - Predicting displacement, strain, and stress fields.
 - Handling nonlinear systems where the small strain assumption becomes invalid and material nonlinearities.
 - Solving inverse problems and problems with complex geometries that otherwise might have been intractable.

Advantages

- Mesh Independence: Avoids issues like mesh quality and distortion seen in FEM.
- Data Scarcity: Incorporates physics laws into training, reducing reliance on large datasets.
- Automatic Differentiation: Computes partial derivatives analytically, improving accuracy.
- Ease of Implementation: Open-source libraries (e.g., TensorFlow) simplify coding.



Governing Equations

1. Equilibrium

$$\sigma_{\alpha\beta,\beta} + f_{\alpha} = 0$$

2. Stress-Strain

$$\sigma_{\alpha\beta} = \lambda \delta_{\alpha\beta} \epsilon_{\gamma\gamma} + 2\mu \epsilon_{\alpha\beta}$$

$$\epsilon_{\alpha\beta} = \frac{1}{2}(u_{\alpha,\beta} + u_{\beta,\alpha})$$

Paper 4 continued

3 Introductory Papers to PINNs

Numerical Examples

- **1D Stretching Rod Problem:**
 - Analytical solution: $u(x)=x/E$
 - Collocation loss achieves RMS error of $5.20E-10$, while energy-based loss achieves $1.80E-3$
- **2D Stretching Plate Problem:**
 - Collocation loss captures stress concentration accurately; energy-based loss shows oscillations in stress fields.

Loss Functions for PINNs

1. Collocation Loss Function

- Minimizes residuals from governing equations and boundary conditions:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (\sigma_{\alpha\beta}^i + f_{\alpha}^i)^2 + \frac{1}{m_t} \sum_{i=1}^{m_t} (\sigma_{\alpha\beta}^i n_{\beta} - \bar{t}_{\alpha}^i)^2$$

- accurate but computationally expensive.

2. Energy-Based Loss Function

- Based on the principle of minimum potential energy:

$$\mathcal{L} = E_{\text{in}} - E_{\text{ex}} \quad E_{\text{in}} = \int_{\Omega} \frac{1}{2} \sigma_{\alpha\beta} \epsilon_{\alpha\beta} dV \quad E_{\text{ex}} = \int_{\Gamma_t} u_{\alpha} \bar{t}_{\alpha} d\Gamma_t$$

- Computationally efficient but less accurate for stress fields.

Currently

4 Current Progress

- I am currently working on recreating a 1D bar with dirichlet and neumann conditions using PINNs used in paper 4.